

# SUBSTR: specify the positions

Make a hidden translation choice visible

01 / THE SAS EXAMPLE

```
data test;
  length text $20 first_three $3
         middle $4 from_seventh $14;
  input text :$20.;
  first_three = substr(text, 1, 3);
  middle = substr(text, 5, 4);
  from_seventh = substr(text, 7);
  datalines;
ABCDEFGHIJ
HelloWorld
SAS-to-Python
;
run;

proc print data=test noobs;
run;
```

**REPORTED TEST SETUP / 21 SEP 2026**  
Claude Sonnet 4.5 / AWS Bedrock GovCloud  
Claude Code 2.1.278 / effortLevel=high / prompt v0.1.0

WHAT YOU WILL PRACTICE

**Use concrete examples to check a translation rule instead of judging code by appearance.**

Extract the beginning, middle, and remainder of each string. SAS positions start at 1; Python slice positions start at 0. Check each result character by character.

EXPECTED RESULT

| Row | first_three | middle | from_seventh |
|-----|-------------|--------|--------------|
| 1   | ABC         | EFGH   | GHIJ         |
| 2   | Hel         | oWor   | orld         |
| 3   | SAS         | to-P   | -Python      |

The table hides trailing storage blanks for readability. Stored widths are 20 for text and 3, 4, 14 for the extracted columns.

Expected values checked in Python; SAS execution pending.

**REPORTED FULL MATCHES**  
**Prompt A: 0/3      Prompt B: 3/3**

ATTACH SOURCE OR PASTE IT AFTER THE PROMPT. START FRESH EACH TIME.

# Prompt A: leave choices open

High degrees of freedom / Notice the decisions the model makes for you.

COPY THIS PROMPT + THE SAS EXAMPLE

```
Convert this SAS code to Python.  
  
Show code in this response. Do not create or run files,  
or read existing Python files.
```

REPORTED TRIAL RESULTS / 21 SEP 2026

0/3 runs matched every expected row

| Run | Row 1 | Row 2 | Row 3 | Full match |
|-----|-------|-------|-------|------------|
| A1  | Yes   | No    | Yes   | No         |
| A2  | Yes   | No    | No    | No         |
| A3  | Yes   | No    | No    | No         |

All three chose pandas bracket slicing (.str[start:stop]).  
No right-padding or assertions were reported.  
A1: CSV row flags show 2/3 rows matched; the narrative summary says 1/3. This table follows the CSV.

SOURCES: results-20260921.csv; trial-log-20260921.md; analysis-20260921.md

WHY CHOICES REMAIN OPEN

- 1

**The code has the rule**

Start positions and lengths are in the SAS source. The model still has to translate them correctly.
- 2

**The method is open**

Bracket slices and .str.slice() can both be correct. A shared method does not prove shared behavior.
- 3

**Appearance is a weak check**

All A runs matched row 1. Checking only ABC would miss the reported failures elsewhere.

SAME INPUT, DIFFERENT REPORTED VALUES

SAS-to-Python / middle / expected: to-P

RUN A2

-to-

RUN A3

o-Py

ANTI-PATTERN: ACCEPTING A PLAUSIBLE TABLE

Execute the returned code and compare every value. Right-padding preserves widths; it does not fix a shifted start position.

# Prompt B: make the choices explicit

Low degrees of freedom / State the behavior, then ask for a check.

COPY THIS PROMPT + THE SAME SAS EXAMPLE

```
Convert this SAS code to Python using pandas.
- Build a DataFrame with text in the supplied row order.
- Right-pad text with spaces to width 20 before slicing.
- Use .str.slice(). Subtract 1 from the SAS start position.
  Python stop = adjusted start + requested length.
- Create first_three with slice(0, 3), middle with slice(4, 8),
  and from_seventh with slice(6, None).
- Preserve result widths 3, 4, 14; trim trailing spaces only
  in the printed view, not in the stored columns.
- Include assertions for the widths and these visible triples:
  ABCDEFGHIJ: ABC, EFGH, GHIJ
  HelloWorld: Hel, oWor, orld
  SAS-to-Python: SAS, to-P, -Python
- Print all columns and briefly explain the position adjustment.

Show code in this response. Do not create or run files,
or read existing Python files.
```

SAS REFERENCE

Evidence: six fresh parallel subagents, three per prompt; raw transcripts were not supplied.  
Assertions passing is reported, not independently rerun here. Python reference checked; SAS run pending.

WHY THESE DETAILS HELP

- 1 Translate the rule**  
SAS start - 1 = Python start. Add the requested length to get the exclusive stop.
- 2 Separate value and display**  
Preserve stored widths 20, 3, 4, 14. Trim trailing spaces only in the printed view.
- 3 Check every slice**  
Assert all nine visible strings and column widths. B1-B3 report padding and passing assertions.

REPORTED RESULTS / 3 OF 3 FULL MATCHES

| Run | Row 1 | Row 2 | Row 3 | Full match |
|-----|-------|-------|-------|------------|
| B1  | Yes   | Yes   | Yes   | Yes        |
| B2  | Yes   | Yes   | Yes   | Yes        |
| B3  | Yes   | Yes   | Yes   | Yes        |

All three used pandas .str.slice().

CHECK IT YOURSELF  
Execute the code; verify values and widths.  
3/3 agreement is not a guarantee of determinism.

# One behavior, three implementations

One behavior contract, three implementations. Run the companion files and the same three acceptance tests against shared fixture data.

SAS / SOURCE BEHAVIOR

```
%macro translate(in=, out=);
data &out;
  length text $20 first_three $3
           middle $4 from_seventh $14;
  set &in;
  first_three = substr(text, 1, 3);
  middle = substr(text, 5, 4);
  from_seventh = substr(text, 7);
run;
%mend;
```

SAS starts at position 1 and takes a length. Explicit storage widths are 20, 3, 4 and 14. CSV loader retains quoted spaces.

PYTHON / PANDAS

```
def translate(df):
    out = df.copy(deep=True)
    out["text"] = out["text"].str.ljust(20)
    s = out["text"].str
    out["first_three"] = s.slice(0, 3)
    out["middle"] = s.slice(4, 8)
    out["from_seventh"] = s.slice(6, None)
    return out
```

pandas slices start at 0 and exclude stop. Pad text to 20 before slicing; keep padding in the stored result columns.

R / BASE

```
translate <- function(df) {
  out <- df
  out$text <- sprintf("%-20s", out$text)
  s <- out$text
  out$first_three <- substr(s, 1, 3)
  out$middle <- substr(s, 5, 8)
  out$from_seventh <- substr(s, 7, 20)
  out
}
```

Base R substr starts at 1 and includes stop: SAS start 5, length 4 becomes R 5 through 8. sprintf pads these ASCII strings.

**Start plus length is not stop**  
SAS substr(text,5,4), Python slice(4,8) and R substr(text,5,8) select the same four positions.

**Stored blanks remain data**  
Input text occupies 20 positions; outputs occupy 3, 4 and 14. Short text still yields padded slices. Trim only a separate display.

SAS SUBSTR

pandas slice

R substr

# Three checks that earn the pass

Use the same inputs and expected behavior in each language. Check the contract as well as the values.

## T1 / Original three-row fixture

INPUT / CONDITION

ABCDEFGH IJ, HelloWorld and SAS-to-Python, in original order.

EXPECTED

Visible triples: ABC/EFGH/GHIJ; Hel/oWor/orld; SAS/to-P/-Python, with exact stored padding.

WHAT THIS CATCHES

Off-by-one starts, confusing length with stop, and losing the remainder.

## T2 / Added short/full-width boundary

INPUT / CONDITION

Added validation inputs AB and 0123456789ABCDEFGHIJ (20 characters).

EXPECTED

AB: "AB ", 4 blanks, 14 blanks. Full-width: "012", "4567", "6789ABCDEFGHIJ".

WHAT THIS CATCHES

Slicing before padding, clipping the last character, or wrong inclusive/exclusive endpoint.

## T3 / Preservation, widths and order

INPUT / CONDITION

All shared rows reversed; original caller data saved.

EXPECTED

Caller unchanged; returned text width 20 and outputs 3/4/14; metadata and row order retained.

WHAT THIS CATCHES

Trimming stored results, variable output widths, caller mutation or row reordering.

REFERENCE CODE + TESTS: `module-02-substr/appendix/`

Run: `python tools/run_appendix_tests.py --modules 02`

New reference code, separate from historical trials. SAS checks await execution; scope is stated in the companion README.